



[nextwork.org](https://nextwork.org)

# Secure Packages with CodeArtifact



sahini sourya

Nextwork-DevOps-CICD [Info](#)

Delete repository

Apply repository policy

Edit

Repository

 this is repository to install packages and dependencies as a part of nextworks CI/CD pipeline project

► Details

Domain, policy, tags, ARN, and upstream repositories.

Packages [Info](#)

↻

Delete package

View connection instructions

🔍

 Filter by package name prefix, format, namespace prefix, and origin controls

<

1

2

3

>

🔍

|                       | Package name                             | Namespace                | Format | Latest version | Latest publish date | Publish | Upstream |
|-----------------------|------------------------------------------|--------------------------|--------|----------------|---------------------|---------|----------|
| <input type="radio"/> | <a href="#">backport-util-concurrent</a> | backport-util-concurrent | maven  | 3.1            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">classworlds</a>              | classworlds              | maven  | 1.1            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">google</a>                   | com.google               | maven  | 1              | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">jsr305</a>                   | com.google.code.findbugs | maven  | 2.0.1          | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">google-collections</a>       | com.google.collections   | maven  | 1.0            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">commons-cli</a>              | commons-cli              | maven  | 1.0            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">commons-logging-api</a>      | commons-logging          | maven  | 1.1            | 10 hours ago        | Block   | Allow    |



# Introducing Today's Project!

In this project, I will demonstrate how to set up code artifact as for CI/CD pipeline's artifact repository!, I'm doing this project to learn the importance and value of having an artifact repository as it a huge time saver.

## Key tools and concepts

Services I used were codeArtifact, IAM, Amazon EC2, Github and VScode. Key concepts I learnt include using artifact repositories, connecting maven with codeArtifact, setting up IAM permissions to give our EC2 instance the permission to access codeArtifact.

## Project reflection

This project took me approximately 3 hours including setup, and trouble shooting. The most challenging part was creating my own package and publish it to codeArtifact repository, using CLI commands and ran into few errors around file names and permission settings. It was most rewarding to see codeArtifact fillup heaps of packages when the connection was all setup.

This project is part three of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tommorow.



# CodeArtifact Repository

CodeArtifact is an artifact repository service, which means i can use it to create repositories to store my web application's packages and dependencies for example a software package you're building an application, you typically use dozens of external packages or libraries - things other developers have created that you don't want to build from scratch. Engineering teams use artifact repositories because for security as Everyone in a team retrieves packages from a secure repository (CodeArtifact), instead of downloading from unsafe sources on the internet, control and reliability like teams can easily share and use the same versions of packages, instead of everyone working with a different version of the same package.

A domain is like a folder that holds together multiple repositories under the same project or under the same organizations. they are helpful to setting up permission settings for multiple codeArtifact repositories in one go. My domain is called nextwork.

A CodeArtifact repository can have an upstream repository, which means a public source of packages that maven can visit if they can't find in local codeArtifact repository. My repository's upstream repository is maven central store, which is largest java repository and is extremely helpful building a java web application.



Developer Tools > CodeArtifact > Repositories > Create repository

Step 1  
Create repository

Step 2  
Select domain

Step 3  
**Review and create**

### Review and create Info

**Package flow**  
Review how packages flow from external connections through nextwork to Nextwork-DevOps-CICD.

| External connections | Domain: <b>nextwork</b>                                     |
|----------------------|-------------------------------------------------------------|
| public:maven-central | Upstream repository <small>?</small><br>maven-central-store |
|                      | Repository<br>Nextwork-DevOps-CICD                          |

```
graph LR; A[public:maven-central] --> B[maven-central-store]; B --> C[Nextwork-DevOps-CICD];
```





# CodeArtifact Security

## Issue

To access CodeArtifact, I need EC2 instance to have the permissions to access codeArtifact's repositories. I need an authentication token that allows my EC2 instance to access codeArtifact for 12 hours. I ran into an error when retrieving a token because my EC2 instance doesn't have the permission to access my AWS resources by default. This follows the principle of least privilege.

## Resolution

To resolve the error with my security token, I setup IAM policy that grants access to codeArtifact and then IAM role that we can apply to EC2 instance. This resolved the error because my EC2 instance now has access to request authorization token from our repository.

It's a security best practice to use IAM roles because IAM roles give the advantage of more secure and scalable instead of hardcoded credentials. As hardcoded credentials are much more vulnerable to security attacks and getting misused (results in errors, losses, delays) here we have IAM roles which are acquired via AWS and do not require any hardcoded credentials which is much more secure and encrypted. IAM roles are the better option if you want to give the same set of permissions for multiple instances.



## The JSON policy attached to my role

The JSON policy I set up grants access to codeArtifact, specifically, it grants access to three key actions related to codeArtifact - retrieving an authorization token, finding the repository[endpoint], viewing the packages inside the repository.

The screenshot shows the AWS IAM Policy Editor interface. On the left, a sidebar indicates the current step is 'Specify permissions'. The main area is titled 'Specify permissions' and includes a sub-header 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' Below this, the 'Policy editor' tab is active, displaying a JSON policy document. The policy is a 'Versioned policy' with a version of '2012-10-17'. It contains two statements: the first is an 'Allow' statement for the actions 'codeartifact:GetAuthorizationToken', 'codeartifact:GetRepositoryEndpoint', and 'codeartifact:ReadFromRepository' on the resource '\*'; the second is an 'Allow' statement for the action 'sts:GetServiceBearerToken' on the resource '\*', with a condition that the 'sts:AWSServiceName' must be 'codeartifact.amazonaws.com'. On the right side of the editor, there is an 'Edit statement' panel with a 'Select a statement' section and an '+ Add new statement' button.

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "codeartifact:GetAuthorizationToken",
8         "codeartifact:GetRepositoryEndpoint",
9         "codeartifact:ReadFromRepository"
10      ],
11       "Resource": "*"
12     },
13     {
14       "Effect": "Allow",
15       "Action": "sts:GetServiceBearerToken",
16       "Resource": "*",
17       "Condition": {
18         "StringEquals": {
19           "sts:AWSServiceName": "codeartifact.amazonaws.com"
20         }
21       }
22     }
23   ]
24 }
```



## Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file configures Maven to use my codeArtifact repository, it supplies maven with the name of authentication token to get access to the codeArtifact repository, and also sets up a profile section in case i've ever had multiple repositories and i need to tell maven when to use which one.

Compiling means a process of translation our web app source code into something machines can run, a maven is our compiler so we asked maven to compile our web app code, while compiling maven will need to put together all the packages that my web app needs in order to run. To retrieve these packages, maven visits CodeArtifact (who will say it doesn't have any packages available), so then Maven requests the exact packages from upstream repository (maven central), once maven has those packages, store local copies in the codeArtifact repository.



The screenshot shows an IDE with a project named 'NEXTWORK-WEB-PROJECT'. The Explorer panel on the left shows the project structure: src/main, resources, webapp, WEB-INF, index.jsp, pom.xml, README.md, and settings.xml. The settings.xml file is open in the editor, showing the following XML content:

```
1 <settings>
2   <servers>
3     <server>
4       <id>nextwork-Nextwork-DevOps-CICD</id>
5       <username>aws</username>
6       <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
7     </server>
8   </servers>
9   <profiles>
10    <profile>
11      <id>nextwork-Nextwork-DevOps-CICD</id>
12      <activation>
13        <activeByDefault>true</activeByDefault>
14      </activation>
15      <repositories>
16        <repository>
17          <id>nextwork-Nextwork-DevOps-CICD</id>
18          <url>https://nextwork-755687317248.d.codeartifact.ap-south-1.amazonaws.com/ma
19        </repository>
20      </repositories>
21    </profile>
22  </profiles>
23  <mirrors>
24    <mirror>
25      <id>nextwork-Nextwork-DevOps-CICD</id>
26      <name>nextwork-Nextwork-DevOps-CICD</name>
27      <url>https://nextwork-755687317248.d.codeartifact.ap-south-1.amazonaws.com/ma
```



## Verify Connection

After compiling, I checked our codeArtifact repository and I noticed 3 pages of packages inside! this tells us that i successfully stored my web app dependencies in a codeArtifact repository.

### Nextwork-DevOps-CICD Info

Delete repositoryApply repository policyEdit

**Repository** this is repository to install packages and dependencies as a part of nextworks CI/CD pipeline project

► **Details**

Domain, policy, tags, ARN, and upstream repositories.

#### Packages Info

↻Delete packageView connection instructions

< 1 2 3 >

⚙

|                       | Package name                             | Namespace                | Format | Latest version | Latest publish date | Publish | Upstream |
|-----------------------|------------------------------------------|--------------------------|--------|----------------|---------------------|---------|----------|
| <input type="radio"/> | <a href="#">backport-util-concurrent</a> | backport-util-concurrent | maven  | 3.1            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">classworlds</a>              | classworlds              | maven  | 1.1            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">google</a>                   | com.google               | maven  | 1              | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">jsr305</a>                   | com.google.code.findbugs | maven  | 2.0.1          | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">google-collections</a>       | com.google.collections   | maven  | 1.0            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">commons-cli</a>              | commons-cli              | maven  | 1.0            | 10 hours ago        | Block   | Allow    |
| <input type="radio"/> | <a href="#">commons-logging-api</a>      | commons-logging          | maven  | 1.1            | 10 hours ago        | Block   | Allow    |



## Uploading My Own Packages

In a project extension, I also decided to become a package publisher which means publish my own packages into my code repository. This is useful in situations where i might have internal team memebbers develpoing thier own packages and wanting thier fellow team mates to have access to it without giving th entire world access.

To create my own package, I set up a text file and use tar to package up the text file. I also generated a security hash because that'll give codeArtifact a way to figure out whether or not package has been tampered with in transit, if it is tampered with, the security hash that codeArtifact has generated will not matech security hash that i generate.

To publish the package, I run a CLI command that let me upload the package which i created in cloud shell to my repository. When I look at the package details in CodeArtifact, I can see version number, published date and even the origin in this case the codeArtifact repository is the origin itself.

To validate my packages, I then try to download my package in cloud shell terminal. I saw my package successfully downloaded, installed from codeArtifact and then i unzipped the package and read what was inside, i could see the very text from inside text file is same before we packaged it up. This proves me the codeArtifact successfully stored custom packages that we can retireve later on aswell.



```
~$ aws codeartifact publish-package-version --domain nextwork --repository NextWork-DevOps-CICD --format generic --namespace secret-mission --package agent-package --package-version 1.0.0 --asset-content secret-mission.tar.gz --asset-name secret-mission.tar.gz --asset-sha256 $ASSET_SHA256
{
  "format": "generic",
  "namespace": "secret-mission",
  "package": "agent-package",
  "version": "1.0.0",
  "versionRevision": "GX1s8Ta7TpeW0woc+H9JTS/cGF4n10xP0aYubH8Mnqk=",
  "status": "Published",
  "asset": {
    "name": "secret-mission.tar.gz",
    "size": 169,
    "hashes": {
      "MD5": "46cfd5a4b669b279d3f65c7a21f7b",
      "SHA-1": "b5ce929abd7bc24343772c63a28587a3e44823",
      "SHA-256": "b6929fc40783c81b12df4c15a279c9181b27e8295f1e918d82c86e25d992c6fe",
      "SHA-512": "429c25237a6341d02304581be1e64f582d8b84023cf0834c972a1e0ffr4fec0b9d35ff702c0bc2349cff05ce5adb9bd8b975f51d7a05c891209cb0c182028b"
    }
  },
  ... skipping ...
  "format": "generic",
  "namespace": "secret-mission",
  "package": "agent-package",
  "version": "1.0.0",
  "versionRevision": "GX1s8Ta7TpeW0woc+H9JTS/cGF4n10xP0aYubH8Mnqk=",
  "status": "Published",
  "asset": {
    "name": "secret-mission.tar.gz",
    "size": 169,
    "hashes": {
      "MD5": "46cfd5a4b669b279d3f65c7a21f7b",
      "SHA-1": "b5ce929abd7bc24343772c63a28587a3e44823",
      "SHA-256": "b6929fc40783c81b12df4c15a279c9181b27e8295f1e918d82c86e25d992c6fe",
      "SHA-512": "429c25237a6341d02304581be1e64f582d8b84023cf0834c972a1e0ffr4fec0b9d35ff702c0bc2349cff05ce5adb9bd8b975f51d7a05c891209cb0c182028b"
    }
  }
}
```



[nextwork.org](https://nextwork.org)

# The place to learn & showcase your skills

Check out [nextwork.org](https://nextwork.org) for more projects

